

Smart Node with Local Alarm and Web View

Build an Arduino based IoT node that monitors its environment, triggers local alarms with LED/buzzer, and exposes its data and status through a small web page served by a Python FastAPI gateway.

Minimum technical requirements (Arduino side)

1. **Use at least one sensor**
 - IMU (acceleration) or DHT20 (temperature/humidity) or both.
2. **Use at least one actuator**
 - LED and/or buzzer.
3. **Implement at least one non blocking state machine**
 - For example: IDLE, MONITORING, ALARM.
4. **Use the button**
 - At least to **start** the program or **acknowledge** an alarm.
 - Keep it simple: short press only.
5. **Send structured data over Serial**
 - One line per “update”, key=value style, e.g.
`T=... ,H=... ,alarm=0/1 ,mode=MONITORING`
 - Include at least: sensor readings and current alarm/mode.
6. **Receive at least one command from Serial**
 - For example:
 - **START** / **STOP** monitoring, or
 - **SUMMARY** → Arduino prints a one line summary, or
 - **RESET** → clears alarm state.

Not allowed to use `delay` for main logic except short beeps / very small pauses inside clearly limited helper functions.

Minimum technical requirements (Python + FastAPI side)

1. **Run FastAPI as a gateway**
 - Background reader from Serial that keeps the latest data in memory (e.g. last 100 lines).
 - `GET /data` that returns JSON with recent measurements and alarm/mode status.
2. **Serve a simple HTML front end**
 - Static `index.html` file.
 - Contains:
 - A small table listing recent data (like they already did for IMU).
 - At least 2 buttons that call FastAPI endpoints.
3. **Implement at least one control endpoint**
 - `POST /start` and/or `POST /stop` or a generic `POST /command`.
 - The endpoint writes a matching command string to Serial.
 - HTML buttons call those endpoints using `fetch`.

Subject 1: “Vibration & Temperature Alarm Node”

Use both IMU and DHT20.

Behavior:

- Every 2 s:
 - Read temperature and humidity, send to PC.
- IMU monitored at higher rate (e.g. 10 Hz):
 - If acceleration magnitude exceeds threshold → mark “shock event”.
- Alarm logic:
 - Alarm triggers if:
 - * Temperature above threshold, **or**
 - * At least N shock events in last M seconds.
 - Alarm shown with LED ON and buzzer beeps (non blocking pattern).
- Button:
 - Short press acknowledges alarm and returns to monitoring mode (alarm resets).
- Serial protocol:
 - Periodic lines with T=...,H=...,shock_count=...,alarm=0/1.
 - Command SUMMARY prints number of shock events and last alarm time.

Web view:

- Table showing:
 - Time, T, H, shock_count, alarm.
 - Buttons:
 - “START” / “STOP” monitoring (forwarded as Serial commands).
-

Subject 2: “Mini Room Climate Monitor”

Use DHT20 only.

Behavior:

- Modes:
 - IDLE: after reset, waiting for button press or START command.
 - MONITORING: sample every 5 s.
 - ALARM: when temperature/humidity out of allowed range.
- In MONITORING:
 - Read T/H every 5 s.
 - If T or H out of bounds:
 - * Enter ALARM, turn LED on, periodic beeps.
- In ALARM:

- Remain until:
 - * Button short press, or
 - * **RESET** command from PC, or
 - * Conditions back to normal for some grace period.
- Serial protocol:
 - Periodic lines: `mode=MONITORING,T=...,H=...,alarm=0/1`.
 - `MODE=IDLE/MONITORING/ALARM` possible in messages.

Web view:

- Display:
 - Latest T/H and mode.
 - Big colored indicator (green = OK, red = alarm).
 - Buttons:
 - “START MONITORING” → send **START**.
 - “RESET ALARM” → send **RESET**.
-

Subject 3: “Motion controlled logger”

Use IMU + button.

Behavior:

- System is **IDLE** until button press or **START** command.
- In **LOGGING** mode:
 - Sample acceleration at 10 Hz, send to PC.
 - If no significant motion (below small threshold) for N seconds, automatically go back to **IDLE**.
- LED:
 - On in **LOGGING**, off in **IDLE**.
- Buzzer:
 - One beep when entering **LOGGING**.
 - Two beeps when auto returning to **IDLE** due to inactivity.

Web view:

- Table of last 100 samples.
- “START LOGGING” / “STOP LOGGING” buttons.
- Optional: display how many seconds since last motion.